

Minimal SI: Defining the International System of Units for the C++ Standard Library

Document #: P4213R0 [[Latest](#)] [[Status](#)]
Date: 2026-05-08
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply-to: Mateusz Pusz
<mateusz.pusz@gmail.com>

Contents

- 1 Introduction
 - 1.1 Motivation
 - 1.1.1 The Content Gap
 - 1.1.2 Why a Separate Paper
 - 1.1.3 Why Modular
 - 1.2 What Is SI?
 - 1.3 The 2019 Redefinition
 - 1.4 Main Components
 - 1.5 Design Overview
 - 1.5.1 Header Naming
 - 1.5.2 Dependency Graph
 - 1.6 C++ Modules
- 2 The International System of Quantities (`<isq_si_quantities>`)
 - 2.1 Base Dimensions
 - 2.2 Base Quantities
 - 2.3 SI-Relevant Derived Quantities
 - 2.3.1 Summary of ISQ Definitions
- 3 Core SI Definitions (`<si_core>`)
 - 3.1 SI Prefixes

- 3.2 SI Units
 - 3.2.1 Base Units
 - 3.2.2 Derived Units with Special Names
 - 3.2.3 Summary of Mandatory Definitions
- 4 Optional: Non-SI Units Accepted for Use with SI (`<si_accepted_units>`)
 - 4.1 Synopsis
 - 4.2 Why This Is a Separate Chapter
- 5 Optional: SI-Defining Constants (`<si_constants>`)
 - 5.1 Versioning by Inline Subnamespace
 - 5.2 Synopsis
 - 5.3 Why This Is a Separate Chapter
- 6 Optional: Unit Symbols (`<si_unit_symbols>`)
 - 6.1 Synopsis (excerpt)
 - 6.2 Symbol Count
 - 6.3 Why This Is a Separate Chapter
- 7 Optional: `std::chrono` Interoperability (`<si_chrono>`)
 - 7.1 Synopsis
 - 7.2 Why This Is a Separate Chapter
- 8 Optional: Mathematical Functions for SI Angles (`<si_math>`)
 - 8.1 Motivation
 - 8.2 Synopsis
 - 8.3 Why This Is a Separate Chapter
- 9 Optional: Prefix Utilities (`<si_prefix_utils>`)
 - 9.1 Motivation
 - 9.2 Synopsis
 - 9.3 Why This Is a Separate Chapter
- 10 Summary
 - 10.1 Definition Counts by Component
- 11 Consolidated Straw Polls
- 12 Acknowledgements
- 13 References

This paper is a companion to [\[P3045R8\]](#) (Quantities and Units Library). Its purpose is to present to LEWG a complete, structured reference of all definitions required to implement

the International System of Units (SI) within a `std::` quantities library, so that LEWG can make informed decisions about scope, phasing, and feasibility of standardization. The synopses are not final interfaces — they reflect the current state of [P3045R8] and [P4185R0] and will evolve as those papers progress. The definitions are organized as a mandatory core (41 ISQ + 81 SI = 122 names) plus six independently-votable optional chapters (~756 additional names), totalling ~878 names across `std::isq` and `std::si`.

§ 1 Introduction

§ 1.1 Motivation

§ 1.1.1 The Content Gap

[P3045R8] supplies the type system. It does not supply the SI. A user who includes the framework gets `quantity`, `unit`, `quantity_spec`, `prefix` — but no `metre`, no `kilogram`, no `second`. The framework is deliberately content-agnostic: it supports arbitrary systems of quantities and units. Shipping the framework without SI content would be like shipping `<algorithm>` without `<vector>` — or the coroutine framework without `std::task`.

§ 1.1.2 Why a Separate Paper

The separation between framework and content is deliberate. [P3045R8] defines the machinery — quantity types, unit types, representation, concepts, text output. This paper defines the domain content — the ISQ quantities needed to classify SI units, the SI units themselves, the prefixes, and the unit symbols — that populates `std::si` and `std::isq` when a user writes `#include <si_core>`.

Separating content from framework enables independent review velocity — LEWG can poll SI definitions without re-reading the type system, and revisions to either paper do not block the other. It also reflects a broader roadmap: SI is the most important system of units but not the only one. Additional papers may follow covering further unit systems and ISQ parts. A paper on mathematical functions for quantities is anticipated, and other library extensions may follow. Each is an independent building block, deliverable whenever [P3045R8] is accepted, without blocking or being blocked by the others.

§ 1.1.3 Why Modular

Not every user needs 715+ unit symbols. Not every embedded target wants trigonometric overloads. Not every codebase interoperates with `std::chrono`. The six optional chapters are independently useful, independently testable, and independently votable.

[*Note:* All synopses in this paper are derived from the open-source `mp-units` library, adapted to use `std::` namespaces. The intent is to show the scope of what standardization requires, not to mandate any particular implementation strategy. — *end note*]

[*Note*: The synopses assume that two features proposed in [\[P4185R0\]](#) are accepted:

- **Non-negativity**: if not accepted, all `non_negative` tags in `quantity_spec` definitions can simply be dropped.
- **Absolute quantities**: if not accepted, `kelvin` must revert to the [\[P3045R8\]](#) design — declaring `absolute_zero` as an `absolute_point_origin` and passing it as the explicit point origin of the unit.

— *end note*]

§ 1.2 What Is SI?

The **International System of Units** (SI) is the modern metric system and the world’s predominant measurement framework, adopted universally in science, engineering, commerce, and everyday life. It is maintained by the International Bureau of Weights and Measures (BIPM) and defined in the SI Brochure [\[SI\]](#).

SI is built on seven **base units** corresponding to the seven base quantities of the International System of Quantities (ISQ):

ISQ quantity	SI unit	Symbol
time	second	s
length	metre	m
mass	kilogram	kg
electric current	ampere	A
thermodynamic temperature	kelvin	K
amount of substance	mole	mol
luminous intensity	candela	cd

All other SI units are derived from these seven by multiplication and division. Twenty-two derived units have been given special names and symbols (newton, pascal, joule, watt, etc.). SI also standardizes **24 unit prefixes** ranging from *quecto* (10^{-30}) to *quetta* (10^{30}), and a set of **non-SI units accepted for use with SI** (minute, hour, litre, electronvolt, etc.).

§ 1.3 The 2019 Redefinition

Before 2019, several SI base units were defined by physical artifacts or experimentally determined constants. The kilogram, for example, was defined by the mass of a platinum-iridium cylinder kept at the BIPM in Sèvres, France.

On 20 May 2019, a landmark redefinition came into effect: all seven SI base units are now derived by fixing the numerical values of seven **defining constants**:

Constant	Symbol	Exact value
Hyperfine transition frequency of Cs-133	$\Delta\nu_{\text{Cs}}$	9 192 631 770 Hz

Constant	Symbol	Exact value
Speed of light in vacuum	c	299 792 458 m/s
Planck constant	h	$6.626\,070\,15 \times 10^{-34}$ J·s
Elementary charge	e	$1.602\,176\,634 \times 10^{-19}$ C
Boltzmann constant	k	$1.380\,649 \times 10^{-23}$ J/K
Avogadro constant	N_A	$6.022\,140\,76 \times 10^{23}$ mol ⁻¹
Luminous efficacy	K_cd	683 lm/W

These values are now exact by definition — there is no experimental uncertainty. The 2019 redefinition makes SI more stable and reproducible.

For a C++ library, this versioning is significant: the numerical values of the defining constants are specific to the 2019 edition. A future SI revision could alter them. A well- designed library should therefore make the SI edition explicit in the API (see [SI-Defining Constants](#)).

§ 1.4 Main Components

The following components are covered by this paper, organized by namespace.

ISQ (`std::isq` namespace) — independent of any unit system:

1. **Base dimensions** — the 7 base dimensions (`dim_length` , `dim_mass` , `dim_time` , `dim_electric_current` , `dim_thermodynamic_temperature` , `dim_amount_of_substance` , `dim_luminous_intensity`) that form the dimensional foundation of the system.
2. **Base quantities** — the 7 base quantities (`length` , `mass` , `duration` , `electric_current` , `thermodynamic_temperature` , `amount_of_substance` , `luminous_intensity`) corresponding to the 7 SI base units.
3. **SI-relevant derived quantities** — the derived quantities required to correctly classify all SI named units: geometry (`width` , `radius` , `path_length` , `area` , `period_duration` , `angular_measure` , `solid_angular_measure` , `frequency`), mechanics (`energy` , `force` , `pressure`), electromagnetism (`electric_potential` , `capacitance` , `impedance` , `admittance` , `magnetic_flux_density`), light and radiation (`luminous_flux` , `illuminance`), physical chemistry (`catalytic_activity`), and atomic/nuclear physics (`activity` , `absorbed_dose` , `dose_equivalent`).

SI core (`std::si` namespace) — depends on ISQ:

1. **Prefixes** — 24 multiplicative scale factors from quecto to quetta.
2. **Base units** — 7 named units plus the special treatment of the kilogram (which already carries a prefix and must therefore be derived from gram).

3. **Derived units with special names** — 22 coherent derived units such as newton and joule, plus the degree Celsius which requires a temperature point origin.

SI optional (`std::si` and related namespaces) — each independently votable:

1. **Non-SI units accepted for use with SI** — time units (minute, hour, day), angular units (degree, arcminute, arcsecond), and others; placed in a dedicated namespace (name open to LEWG discussion) and re-exported into `std::si`.
2. **SI-defining constants** — the 7 exact constants plus a small number of commonly needed derived constants; versioned by inline subnamespace.
3. **Unit symbols** — short-form aliases (`m` , `km` , `Hz` , ...) for all prefix-unit combinations.
4. **`std::chrono` interoperability** — conversion between SI time quantities and `std::chrono::duration` / `std::chrono::time_point`.
5. **Trigonometric functions for SI angles** — `sin` , `cos` , `tan` , `asin` , `acos` , `atan` , `atan2` overloads that accept and return `quantity_of<isq::angular_measure>`.
6. **Prefix utilities** — an `invoke_with_prefixed` facility for automatically selecting the most appropriate SI prefix when formatting a quantity for display.

§ 1.5 Design Overview

§ 1.5.1 Header Naming

Headers use a flat, underscore-separated naming convention — for example `<si_core>` , `<si_constants>` , `<isq>` — rather than a slash-based hierarchy. A slash-based scheme would require both a file named `si` (for an aggregate header) and a directory named `si/` (for sub-headers) to coexist, which is impossible on POSIX filesystems when header names map directly to file paths.

Three aggregate headers span the full scope:

- `<isq_si_quantities>` — the minimal ISQ subset for SI; lives in `std::isq` and is independent of any unit system. Other unit systems may depend on it without taking a dependency on `<si_core>`.
- `<isq>` — aggregate of all ISQ headers; will grow as ISO/IEC 80000 parts are standardized. `<si_core>` does not depend on it.
- `<si>` — includes all SI headers: `<si_core>` , `<si_accepted_units>` , `<si_constants>` , `<si_unit_symbols>` , `<si_chrono>` , `<si_math>` , and `<si_prefix_utils>`.

§ 1.5.2 Dependency Graph

The header dependency structure is a simple tree rooted at the P3045 framework:

P3045 (Quantities and Units Library framework)

└─ <code><isq_si_quantities></code>	mandatory · freestanding · 41 names
└─ <code><si_core></code>	mandatory · freestanding · 81 names
└─ <code><si_accepted_units></code>	optional · freestanding
└─ <code><si_constants></code>	optional · freestanding
└─ <code><si_unit_symbols></code>	optional · freestanding
└─ <code><si_chrono></code>	optional · hosted (requires <code><chrono></code>)
└─ <code><si_math></code>	optional · hosted (requires <code><cmath></code>)
└─ <code><si_prefix_utils></code>	optional · hosted (requires <code><cmath></code>)

Convenience aggregates (each includes all headers in its subtree):

`<isq>` → `<isq_si_quantities>` and future ISQ part headers
`<si>` → `<si_core>` and all optional SI headers above

Every optional header depends directly on `<si_core>`. No optional header depends on another optional header.

LEWG should consider voting on `<isq_si_quantities>` (§2), `<si_core>` (§3), and each optional chapter (§4–§9) separately.

The remainder of this paper presents the C++ synopsis for each component, together with a definition count and a rationale for its classification as mandatory or optional.

§ 1.6 C++ Modules

Since C++23, the entire standard library is exposed as a single module — `import std;`. All SI and ISQ definitions would simply be part of that aggregate, and no separate module hierarchy is needed.

If a future standard wishes to introduce finer-grained modules, they could look like:

```
import std.si;           // all SI definitions
import std.isq;          // all ISQ definitions
```

§ 2 The International System of Quantities (`<isq_si_quantities>`)

ISQ (ISO 80000) defines physical quantities independently of any unit system; the full standard spans over 10 parts covering hundreds of quantities. This paper standardizes only the minimal subset needed to define SI: seven base dimensions, seven base quantities, and the derived quantities required to correctly classify the 22 SI coherent derived units. All of these reside in `std::isq` and are provided by a single header, `<isq_si_quantities>`.

§ 2.1 Base Dimensions

```
namespace std::isq {

inline constexpr struct dim_length : base_dimension<"L"> {} dim_length;
inline constexpr struct dim_mass : base_dimension<"M"> {} dim_mass;
```

```

inline constexpr struct dim_time : base_dimension<"T"> {} dim_time;
inline constexpr struct dim_electric_current : base_dimension<"I"> {}
    dim_electric_current;
inline constexpr struct dim_thermodynamic_temperature : base_dimension<symbol_text{u8"θ", "0"}> {} dim_thermodynamic_temperature;
inline constexpr struct dim_amount_of_substance : base_dimension<"N"> {}
    dim_amount_of_substance;
inline constexpr struct dim_luminous_intensity : base_dimension<"J"> {}
    dim_luminous_intensity;

} // namespace std::isq

```

This introduces **7 names** in `std::isq`.

§ 2.2 Base Quantities

```

namespace std::isq {

inline constexpr struct length : quantity_spec<dim_length, non_negative>
    {} length;
inline constexpr struct mass : quantity_spec<dim_mass, non_negative> {}
    mass;
inline constexpr struct duration : quantity_spec<dim_time, non_negative>
    {} duration;
inline constexpr auto time = duration;
inline constexpr struct electric_current : quantity_spec<dim_electric_current> {} electric_current;
inline constexpr struct thermodynamic_temperature : quantity_spec<dim_thermodynamic_temperature, non_negative> {}
    thermodynamic_temperature;
inline constexpr struct amount_of_substance : quantity_spec<dim_amount_of_substance, non_negative> {} amount_of_substance;
inline constexpr struct luminous_intensity : quantity_spec<dim_luminous_intensity, non_negative> {} luminous_intensity;

} // namespace std::isq

```

This introduces **8 names** in `std::isq` (7 base quantity types and a time alias for duration, which is the SI name for the same base quantity).

§ 2.3 SI-Relevant Derived Quantities

The following derived quantities form the minimal ISQ subset needed to correctly classify all 22 SI named derived units:

```

namespace std::isq {

// space and time
inline constexpr struct width : quantity_spec<length> {} width;
inline constexpr auto breadth = width;
inline constexpr struct radius : quantity_spec<width> {} radius;
inline constexpr struct path_length : quantity_spec<length> {}
    path_length;
inline constexpr auto arc_length = path_length;

```

```

inline constexpr struct area : quantity_spec<pow<2>(length), non_negative> {} area;
inline constexpr struct angular_measure : quantity_spec<dimensionless, arc_length / radius, is_kind> {} angular_measure;
inline constexpr struct solid_angular_measure : quantity_spec<dimensionless, area / pow<2>(radius), is_kind, non_negative> {} solid_angular_measure;
inline constexpr struct period_duration : quantity_spec<duration> {} period_duration;
inline constexpr auto period = period_duration;
inline constexpr struct frequency : quantity_spec<inverse(period_duration), non_negative> {} frequency;

// mechanics
inline constexpr struct energy : quantity_spec<mass * pow<2>(length) / pow<2>(duration), non_negative> {} energy;
inline constexpr struct force : quantity_spec<mass * length / pow<2>(duration), quantity_character::vector> {} force;
inline constexpr struct pressure : quantity_spec<force / area, quantity_character::real_scalar> {} pressure;

// electromagnetism
inline constexpr struct electric_potential : quantity_spec<energy / (electric_current * duration), quantity_character::real_scalar> {} electric_potential;
inline constexpr struct capacitance : quantity_spec<electric_current * duration / electric_potential, non_negative> {} capacitance;
inline constexpr struct impedance : quantity_spec<electric_potential / electric_current, quantity_character::complex_scalar> {} impedance;
inline constexpr struct admittance : quantity_spec<inverse(impedance), quantity_character::complex_scalar> {} admittance;
inline constexpr struct magnetic_flux_density : quantity_spec<mass / (electric_current * pow<2>(duration)), quantity_character::vector> {} magnetic_flux_density;

// light and radiation
inline constexpr struct luminous_flux : quantity_spec<luminous_intensity * solid_angular_measure, non_negative> {} luminous_flux;
inline constexpr struct illuminance : quantity_spec<luminous_flux / area, non_negative> {} illuminance;

// physical chemistry
inline constexpr struct catalytic_activity : quantity_spec<amount_of_substance / duration, non_negative> {} catalytic_activity;

// atomic and nuclear physics
inline constexpr struct activity : quantity_spec<inverse(duration), non_negative> {} activity;
inline constexpr struct absorbed_dose : quantity_spec<energy / mass, non_negative> {} absorbed_dose;
inline constexpr struct ionizing_radiation_quality_factor : quantity_spec<dimensionless, non_negative> {} ionizing_radiation_quality_factor;
inline constexpr struct dose_equivalent

```

```

        : quantity_spec<absorbed_dose * ionizing_radiation_quality_factor,
          non_negative> {} dose_equivalent;

} // namespace std::isq

```

This introduces **26 names** in `std::isq` (23 derived quantity types and 3 aliases: `breadth`, `arc_length`, and `period`).

§ 2.3.1 Summary of ISQ Definitions

Component	Names introduced
Base dimensions	7
Base quantities (incl. time alias)	8
SI-relevant derived quantities (incl. aliases)	26
Total	41

Straw poll: We want `<isq_si_quantities>` — the minimal ISQ subset (41 definitions) required by `<si_core>` — to be standardized as part of the quantities and units library.

§ 3 Core SI Definitions (`<si_core>`)

`<si_core>` is the single mandatory SI header. It provides the 24 SI prefixes and all base and derived unit definitions.

§ 3.1 SI Prefixes

The 24 standard SI prefixes are defined as class templates parameterised on a `PrefixableUnit` type, together with corresponding variable templates that provide the user-facing API:

```

namespace std::si {

template<PrefixableUnit U> struct quecto_ : prefixed_unit<"q", mag_power<10, -30>, U{}> {};
template<PrefixableUnit U> struct ronto_ : prefixed_unit<"r", mag_power<10, -27>, U{}> {};
template<PrefixableUnit U> struct yocto_ : prefixed_unit<"y", mag_power<10, -24>, U{}> {};
template<PrefixableUnit U> struct zepto_ : prefixed_unit<"z", mag_power<10, -21>, U{}> {};
template<PrefixableUnit U> struct atto_ : prefixed_unit<"a", mag_power<10, -18>, U{}> {};
template<PrefixableUnit U> struct femto_ : prefixed_unit<"f", mag_power<10, -15>, U{}> {};
template<PrefixableUnit U> struct pico_ : prefixed_unit<"p", mag_power<10, -12>, U{}> {};
template<PrefixableUnit U> struct nano_ : prefixed_unit<"n", mag_power<10, -9>, U{}> {};
template<PrefixableUnit U> struct micro_ : prefixed_unit<symbol_text{u8"μ", "u"}, mag_power<10, -6>, U{}> {};

```

```

template<PrefixableUnit U> struct milli_ : prefixed_unit<"m", mag_pow-
    er<10, -3>, U{}> {};
template<PrefixableUnit U> struct centi_ : prefixed_unit<"c", mag_pow-
    er<10, -2>, U{}> {};
template<PrefixableUnit U> struct deci_ : prefixed_unit<"d", mag_pow-
    er<10, -1>, U{}> {};
template<PrefixableUnit U> struct deca_ : prefixed_unit<"da", mag_pow-
    er<10, 1>, U{}> {};
template<PrefixableUnit U> struct hecto_ : prefixed_unit<"h", mag_pow-
    er<10, 2>, U{}> {};
template<PrefixableUnit U> struct kilo_ : prefixed_unit<"k", mag_pow-
    er<10, 3>, U{}> {};
template<PrefixableUnit U> struct mega_ : prefixed_unit<"M", mag_pow-
    er<10, 6>, U{}> {};
template<PrefixableUnit U> struct giga_ : prefixed_unit<"G", mag_pow-
    er<10, 9>, U{}> {};
template<PrefixableUnit U> struct tera_ : prefixed_unit<"T", mag_pow-
    er<10, 12>, U{}> {};
template<PrefixableUnit U> struct peta_ : prefixed_unit<"P", mag_pow-
    er<10, 15>, U{}> {};
template<PrefixableUnit U> struct exa_ : prefixed_unit<"E", mag_pow-
    er<10, 18>, U{}> {};
template<PrefixableUnit U> struct zetta_ : prefixed_unit<"Z", mag_pow-
    er<10, 21>, U{}> {};
template<PrefixableUnit U> struct yotta_ : prefixed_unit<"Y", mag_pow-
    er<10, 24>, U{}> {};
template<PrefixableUnit U> struct ronna_ : prefixed_unit<"R", mag_pow-
    er<10, 27>, U{}> {};
template<PrefixableUnit U> struct quetta_ : prefixed_unit<"Q", mag_pow-
    er<10, 30>, U{}> {};

template<PrefixableUnit auto U> constexpr quecto_<decltype(U)> quecto;
template<PrefixableUnit auto U> constexpr ronto_<decltype(U)> ronto;
template<PrefixableUnit auto U> constexpr yocto_<decltype(U)> yocto;
template<PrefixableUnit auto U> constexpr zepto_<decltype(U)> zepto;
template<PrefixableUnit auto U> constexpr atto_<decltype(U)> atto;
template<PrefixableUnit auto U> constexpr femto_<decltype(U)> femto;
template<PrefixableUnit auto U> constexpr pico_<decltype(U)> pico;
template<PrefixableUnit auto U> constexpr nano_<decltype(U)> nano;
template<PrefixableUnit auto U> constexpr micro_<decltype(U)> micro;
template<PrefixableUnit auto U> constexpr milli_<decltype(U)> milli;
template<PrefixableUnit auto U> constexpr centi_<decltype(U)> centi;
template<PrefixableUnit auto U> constexpr deci_<decltype(U)> deci;
template<PrefixableUnit auto U> constexpr deca_<decltype(U)> deca;
template<PrefixableUnit auto U> constexpr hecto_<decltype(U)> hecto;
template<PrefixableUnit auto U> constexpr kilo_<decltype(U)> kilo;
template<PrefixableUnit auto U> constexpr mega_<decltype(U)> mega;
template<PrefixableUnit auto U> constexpr giga_<decltype(U)> giga;
template<PrefixableUnit auto U> constexpr tera_<decltype(U)> tera;
template<PrefixableUnit auto U> constexpr peta_<decltype(U)> peta;
template<PrefixableUnit auto U> constexpr exa_<decltype(U)> exa;
template<PrefixableUnit auto U> constexpr zetta_<decltype(U)> zetta;
template<PrefixableUnit auto U> constexpr yotta_<decltype(U)> yotta;
template<PrefixableUnit auto U> constexpr ronna_<decltype(U)> ronna;
template<PrefixableUnit auto U> constexpr quetta_<decltype(U)> quetta;

} // namespace std::si

```

This introduces **48 names** in `std::si` (24 class templates + 24 variable templates).

§ 3.2 SI Units

§ 3.2.1 Base Units

```
namespace std::si {  
  
    inline constexpr struct second : named_unit<"s", kind_of<isq::duration>> {} second;  
    inline constexpr struct metre : named_unit<"m", kind_of<isq::length>> {} metre;  
    inline constexpr struct gram : named_unit<"g", kind_of<isq::mass>> {} gram;  
    inline constexpr auto kilogram = kilo<gram>;  
    inline constexpr struct ampere : named_unit<"A", kind_of<isq::electric_current>> {} ampere;  
    inline constexpr struct kelvin : named_unit<"K", kind_of<isq::thermodynamic_temperature>> {} kelvin;  
    inline constexpr struct mole : named_unit<"mol", kind_of<isq::amount_of_substance>> {} mole;  
    inline constexpr struct candela : named_unit<"cd", kind_of<isq::luminous_intensity>> {} candela;  
  
} // namespace std::si
```

This introduces **8 names** in `std::si`. Note that kilogram is defined as `kilo<gram>` rather than as a primary unit. This is by design: because kilogram already carries the kilo prefix, SI prefixes must be applied to gram (e.g., `milli<gram>`, `micro<gram>`). The library models this correctly by treating gram as the prefixable base and deriving kilogram from it.

Under the absolute quantities model from [P4185R0], kelvin requires no explicit point origin. A quantity `<si::kelvin>` is intrinsically absolute — `28 * K` compiles and produces an absolute thermodynamic temperature directly. In the pre-P4185 design ([P3045R8]), kelvin had to declare `absolute_zero` as an `absolute_point_origin`, which prevented the multiply syntax from being used and forced users to write `absolute_zero + 28 * K` instead.

§ 3.2.2 Derived Units with Special Names

SI defines 22 coherent derived units with special names, plus three temperature-related definitions (`absolute_zero`, `ice_point`, `degree_Celsius`):

```
namespace std::si {  
  
    inline constexpr struct radian : named_unit<"rad", metre / metre, kind_of<isq::angular_measure>> {} radian;  
    inline constexpr struct steradian : named_unit<"sr", square(metre) / square(metre), kind_of<isq::solid_angular_measure>> {} steradian;  
    inline constexpr struct hertz : named_unit<"Hz", one / second, kind_of<isq::frequency>> {} hertz;
```

```

inline constexpr struct newton : named_unit<"N", kilogram * metre /
    square(second), kind_of<isq::force>> {} newton;
inline constexpr struct pascal : named_unit<"Pa", newton / square(me-
    tre), kind_of<isq::pressure>> {} pascal;
inline constexpr struct joule : named_unit<"J", newton * metre,
    kind_of<isq::energy>> {} joule;
// No kind_of: W is shared by isq::power, isq::heat_flow_rate, isq::radi-
    ant_flux, etc.
inline constexpr struct watt : named_unit<"W", joule / second> {}
    watt;
// No kind_of: C is shared by isq::electric_charge and isq::electric_flux
    (different ISQ hierarchies).
inline constexpr struct coulomb : named_unit<"C", ampere * second> {}
    coulomb;
inline constexpr struct volt : named_unit<"V", watt / ampere,
    kind_of<isq::electric_potential>> {} volt;
inline constexpr struct farad : named_unit<"F", coulomb / volt,
    kind_of<isq::capacitance>> {} farad;
inline constexpr struct ohm : named_unit<symbol_text{u8"Ω", "ohm"},
    volt / ampere, kind_of<isq::impedance>> {} ohm;
inline constexpr struct siemens : named_unit<"S", one / ohm,
    kind_of<isq::admittance>> {} siemens;
// No kind_of: Wb is shared by isq::magnetic_flux, isq::protoflux,
    isq::linked_magnetic_flux, etc.
inline constexpr struct weber : named_unit<"Wb", volt * second> {}
    weber;
inline constexpr struct tesla : named_unit<"T", weber / square(metre),
    kind_of<isq::magnetic_flux_density>> {} tesla;
// No kind_of: H is shared by isq::inductance and isq::permeance (differ-
    ent ISQ hierarchies).
inline constexpr struct henry : named_unit<"H", weber / ampere> {}
    henry;

inline constexpr struct absolute_zero : relative_point_ori-
    gin<point<kelvin>(0)> {} absolute_zero;
inline constexpr struct ice_point : relative_point_origin<point<mil-
    li<kelvin>>(273'150)> {} ice_point;
inline constexpr struct degree_Celsius : named_unit<symbol_text{u8"°C ",
    "°C"}, kelvin, ice_point> {} degree_Celsius;

inline constexpr struct lumen : named_unit<"lm", candela * steradian,
    kind_of<isq::luminous_flux>> {} lumen;
inline constexpr struct lux : named_unit<"lx", lumen / square(me-
    tre), kind_of<isq::illuminance>> {} lux;
inline constexpr struct becquerel : named_unit<"Bq", one / second,
    kind_of<isq::activity>> {} becquerel;
inline constexpr struct gray : named_unit<"Gy", joule / kilogram,
    kind_of<isq::absorbed_dose>> {} gray;
inline constexpr struct sievert : named_unit<"Sv", joule / kilogram,
    kind_of<isq::dose_equivalent>> {} sievert;
inline constexpr struct katal : named_unit<"kat", mole / second,
    kind_of<isq::catalytic_activity>> {} katal;

} // namespace std::si

```

This introduces **25 names** in `std::si`: the 22 coherent derived units with special names, plus `absolute_zero`, `ice_point`, and `degree_Celsius`.

[*Note*: Four SI named units are deliberately defined without a `kind_of<>` constraint because they serve multiple unrelated ISQ quantity hierarchies. Restricting them to a single kind would incorrectly reject legitimate uses from the other hierarchies.

Unit	ISQ quantity hierarchies that use it
watt (W)	<i>power, heat flow rate, active power, radiant flux</i> — spanning mechanics, thermodynamics, electromagnetism, and radiometry
coulomb (C)	<i>electric charge</i> and <i>electric flux</i> — distinct hierarchies sharing the same dimension
weber (Wb)	<i>magnetic flux, protoflux, linked magnetic flux, total magnetic flux</i>
henry (H)	<i>inductance</i> and <i>permeance</i> — unrelated quantities with the same dimension

— *end note*]

`absolute_zero` is defined as a `relative_point_origin` at `point<kelvin>(0)` — a named origin at 0 K — rather than as an `absolute_point_origin`. Under the absolute quantities model from [P4185R0], the natural zero of `isq::thermodynamic_temperature` is implicit in every kelvin quantity; there is no need for an independent origin declaration. `absolute_zero` is retained as a named convenience for the `temp - absolute_zero` idiom and for explicit zero-point arithmetic. `ice_point` is defined as an offset of 273.150 K from that same natural zero, and `degree_Celsius` is the unit for quantities anchored at `ice_point`.

§ 3.2.3 Summary of Mandatory Definitions

Component	Names introduced
SI prefixes (class templates)	24
SI prefixes (variable templates)	24
SI base units	8
SI named derived units and origins	25
Total	81

Straw poll: We want `<si_core>` — 81 SI definitions including 24 prefixes, 8 base units, and 25 derived units and origins — to be standardized as part of the quantities and units library.

§ 4 Optional: Non-SI Units Accepted for Use with SI (`<si_accepted_units>`)

The SI Brochure recognizes a number of units outside SI proper that are accepted for use alongside it. These are placed in a dedicated namespace and re-exported into `std::si` for convenience.

[*Note:* The `non_si` namespace name used here is a working name. The author does not insist on it — it is introduced to make the separation from core SI units explicit. LEWG is welcome to choose any name it sees fit. — *end note*]

§ 4.1 Synopsis

```
namespace std::non_si {

inline constexpr struct minute      : named_unit<"min", mag<60> * si::second> {} minute;
inline constexpr struct hour        : named_unit<"h",   mag<60> * minute> {} hour;
inline constexpr struct day          : named_unit<"d",   mag<24> * hour> {} day;
inline constexpr struct astronomical_unit : named_unit<"au", mag<149'597'870'700> * si::metre> {} astronomical_unit;
inline constexpr struct degree       : named_unit<symbol_text{u8"°", "deg"}, mag_ratio<1, 180> * π * si::radian> {} degree;
inline constexpr struct arcminute    : named_unit<symbol_text{u8"'", ""}, mag_ratio<1, 60> * degree> {} arcminute;
inline constexpr struct arcsecond    : named_unit<symbol_text{u8"\"", ""}, mag_ratio<1, 60> * arcminute> {} arcsecond;
inline constexpr struct are           : named_unit<"a", square(si::deca<si::metre>)> {} are;
inline constexpr auto hectare = si::hecto<are>;
inline constexpr struct litre        : named_unit<"L",   cubic(si::deci<si::metre>)> {} litre;
inline constexpr struct tonne        : named_unit<"t",   mag<1000> * si::kilogram> {} tonne;
inline constexpr struct dalton        : named_unit<"Da", mag_ratio<16'605'390'666'050, 10'000'000'000'000> * mag_power<10, -27> * si::kilogram> {} dalton;
inline constexpr struct electronvolt : named_unit<"eV", mag_ratio<1'602'176'634, 1'000'000'000> * mag_power<10, -19> * si::joule> {} electronvolt;

} // namespace std::non_si

namespace std::si {
    using namespace non_si;
} // namespace std::si
```

Additionally, for the angular units, the library specializes the `space_before_unit_symbol` customization point to suppress the space between the numerical value and the degree symbol. The SI Brochure [SI] §5.4.3 states that the unit symbols °, ', and " are not preceded by a space:

```
template<> inline constexpr bool space_before_unit_symbol<non_si::degree>
    = false;
template<> inline constexpr bool space_before_unit_symbol<non_si::arcminute>
    = false;
template<> inline constexpr bool space_before_unit_symbol<non_si::arcsecond>
    = false;
```

Category	Count
Named units	13
Customization point specializations	3
Total	16

This introduces **13 names** in `std::non_si` and **3 customization point specializations**.

§ 4.2 Why This Is a Separate Chapter

These units form a natural boundary with SI proper: they are recognized by the SI Brochure as units accepted for use with SI, but they are not SI units. Separating them into a distinct chapter lets LEWG decide on their inclusion independently of the core, and also allows the namespace naming question (`non_si` or another name) to be resolved on its own schedule without blocking the mandatory definitions.

Straw poll: *We want `<si_accepted_units>` — non-SI units accepted for use with SI — to be included as part of the quantities and units library.*

§ 5 Optional: SI-Defining Constants (`<si_constants>`)

`<si_constants>` provides the seven exact constants that define the 2019 SI, plus a small set of derived constants in common use. These constants appear directly in physical equations ($E = mc^2$, Planck’s law, the ideal gas law, etc.) and, because they are named_constant objects, they are unit-like: they embed in the types of quantity expressions. A function returning `quantity<si::speed_of_light_in_vacuum * si::second>` has a type that is only compatible with other code that names the same constant. Standardizing these definitions therefore matters for library interoperability, not only for scientific computing.

§ 5.1 Versioning by Inline Subnamespace

Because the defining constants are specific to the 2019 edition of SI, they are placed in an `inline namespace si2019` inside `std::si`. This achieves two goals simultaneously:

1. **Default access:** `std::si::speed_of_light_in_vacuum` resolves to the 2019 value without any additional qualification, since `si2019` is inline.
2. **Explicit versioning:** Code that must unambiguously refer to the 2019 values can write `std::si::si2019::speed_of_light_in_vacuum`. This form will remain stable even if a future SI revision introduces `inline namespace si20XX` with updated values.

If SI is redefined in the future, the new constants would be placed in `inline namespace si20XX` and the `inline` qualifier would be moved from `si2019` to `si20XX`, making the

new values the default while preserving backward compatibility for code using the versioned form.

§ 5.2 Synopsis

```
namespace std::si {

inline namespace si2019 {

inline constexpr struct hyperfine_structure_transition_frequency_of_cs
    : named_constant<symbol_text{u8"Δv_Cs", "dv_Cs"}, mag<9'192'631'770>
      * hertz> {} hyperfine_structure_transition_frequency_of_cs;

inline constexpr struct speed_of_light_in_vacuum
    : named_constant<"c", mag<299'792'458> * metre / second> {}
    speed_of_light_in_vacuum;

inline constexpr struct planck_constant
    : named_constant<"h", mag_ratio<662'607'015, 100'000'000> * mag_pow-
      er<10, -34> * joule * second> {} planck_constant;

inline constexpr struct elementary_charge
    : named_constant<"e", mag_ratio<1'602'176'634, 1'000'000'000> *
      mag_power<10, -19> * coulomb> {} elementary_charge;

inline constexpr struct boltzmann_constant
    : named_constant<"k", mag_ratio<1'380'649, 1'000'000> * mag_power<10,
      -23> * joule / kelvin> {} boltzmann_constant;

inline constexpr struct avogadro_constant
    : named_constant<"N_A", mag_ratio<602'214'076, 100'000'000> * mag_pow-
      er<10, 23> / mole> {} avogadro_constant;

inline constexpr struct luminous_efficacy : named_constant<"K_cd",
    mag<683> * lumen / watt> {} luminous_efficacy;

} // namespace si2019

// A selection of derived constants found useful in practice – this list
// is not exhaustive.
inline constexpr struct standard_gravity
    : named_constant<symbol_text{u8"g₀", "g_0"}, mag_ratio<980'665,
      100'000> * metre / square(second)> {} standard_gravity;

inline constexpr struct reduced_planck_constant
    : named_constant<symbol_text{u8"ħ", "hbar"}, si2019::planck_constant /
      (mag<2> * π)> {} reduced_planck_constant;

inline constexpr struct magnetic_constant
    : named_constant<symbol_text{u8"μ₀", "u_0"}, mag<4> * mag_power<10, -7>
      * π * henry / metre> {} magnetic_constant;

} // namespace std::si
```

Category	Count
SI-defining constants (<code>si2019</code>)	7
Derived physical constants	3
Total	10

This introduces **10 names** in `std::si`: the 7 SI-defining constants (within `si2019`) plus 3 additional constants (`standard_gravity` , `reduced_planck_constant` , `magnetic_constant`).

[*Note*: The three additional constants listed above are a small, arbitrary selection drawn from the `mp-units` library — constants that proved useful in practice. This list is neither normative nor complete: there are many other derived constants in common use (Faraday constant, Stefan–Boltzmann constant, Rydberg constant, etc.). LEWG should treat the question of which derived constants to include as a separate design decision, independent of whether the seven SI-defining constants are standardized. — *end note*]

§ 5.3 Why This Is a Separate Chapter

The seven defining constants and any derived constants form a distinct kind of vocabulary: they are physical constants, not units, and they raise separate design questions — versioning strategy, which derived constants to include, and how many. Separating them into a dedicated chapter lets LEWG vote on these questions independently of the unit definitions.

Straw poll: *We want the seven SI-defining constants (`si2019` namespace) to be included as part of the quantities and units library.*

Straw poll: *We want a selected set of derived physical constants to be included as part of the quantities and units library.*

§ 6 Optional: Unit Symbols (`<si_unit_symbols>`)

`<si_unit_symbols>` provides short-form variable names for every combination of SI prefix and SI unit. These are the symbols printed in textbooks and used in everyday engineering: m for metre, km for kilometre, Hz for hertz, MHz for megahertz, and so on.

Using unit symbols enables concise, readable quantity expressions:

```
using namespace std::si::unit_symbols;
quantity distance = 100 * km;
quantity time     = 9.58 * s;
quantity speed    = distance / time; // approximately 10.44 km/s
```

The symbols are defined in `std::si::unit_symbols` and `std::non_si::unit_symbols`; `std::si::unit_symbols` re-exports everything from `std::non_si::unit_symbols`.

§ 6.1 Synopsis (excerpt)

The complete synopsis is large. A representative excerpt for the metre is shown below; the same pattern is repeated for every SI base and named derived unit:

```
namespace std::si::unit_symbols {

// metre – all 24 SI prefix multiples, plus Unicode alias for micro
inline constexpr auto qm = quecto<metre>; // 10-30 m
inline constexpr auto rm = ronto<metre>; // 10-27 m
inline constexpr auto ym = yocto<metre>; // 10-24 m
inline constexpr auto zm = zepto<metre>; // 10-21 m
inline constexpr auto am = atto<metre>; // 10-18 m
inline constexpr auto fm = femto<metre>; // 10-15 m
inline constexpr auto pm = pico<metre>; // 10-12 m
inline constexpr auto nm = nano<metre>; // 10-9 m
inline constexpr auto um = micro<metre>; // 10-6 m (ASCII alias)
inline constexpr auto μm = micro<metre>; // 10-6 m (Unicode alias)
inline constexpr auto mm = milli<metre>; // 10-3 m
inline constexpr auto cm = centi<metre>; // 10-2 m
inline constexpr auto dm = deci<metre>; // 10-1 m
inline constexpr auto m = metre; // m
inline constexpr auto dam = deca<metre>; // 10 m
inline constexpr auto hm = hecto<metre>; // 102 m
inline constexpr auto km = kilo<metre>; // 103 m
inline constexpr auto Mm = mega<metre>; // 106 m
inline constexpr auto Gm = giga<metre>; // 109 m
inline constexpr auto Tm = tera<metre>; // 1012 m
inline constexpr auto Pm = peta<metre>; // 1015 m
inline constexpr auto Em = exa<metre>; // 1018 m
inline constexpr auto Zm = zetta<metre>; // 1021 m
inline constexpr auto Ym = yotta<metre>; // 1024 m
inline constexpr auto Rm = ronna<metre>; // 1027 m
inline constexpr auto Qm = quetta<metre>; // 1030 m

// The same pattern is repeated for:
// s (second), g/kg (gram/kilogram), A (ampere), K (kelvin),
// mol (mole), cd (candela), rad (radian), sr (steradian),
// Hz (hertz), N (newton), Pa (pascal), J (joule), W (watt),
// C (coulomb), V (volt), F (farad), Ω/ohm (ohm), S (siemens),
// Wb (weber), T (tesla), H (henry), lm (lumen), lx (lux),
// Bq (becquerel), Gy (gray), Sv (sievert), kat (katal).

// Units with non-ASCII symbols additionally provide Unicode variable
// names:
// Ω alongside ohm, μ alongside u.

// Commonly used squared and cubic forms:
inline constexpr auto m2 = square(metre);
inline constexpr auto m3 = cubic(metre);
inline constexpr auto m4 = pow<4>(metre);
inline constexpr auto s2 = square(second);
inline constexpr auto s3 = cubic(second);

} // namespace std::si::unit_symbols
```

The non-SI unit symbols are in a separate namespace, and re-exported into `si::unit_symbols`:

```
namespace std::non_si::unit_symbols {  
  
    inline constexpr auto au      = astronomical_unit;  
    inline constexpr auto deg     = degree;  
    inline constexpr auto arcmin  = arcminute;  
    inline constexpr auto arcsec  = arcsecond;  
    inline constexpr auto a       = are;  
    inline constexpr auto ha      = hectare;  
    inline constexpr auto l       = litre;  
    inline constexpr auto L       = litre;    // both spellings are permitted by  
        SI  
    inline constexpr auto t       = tonne;  
    inline constexpr auto Da      = dalton;  
    inline constexpr auto eV      = electronvolt;  
    inline constexpr auto min     = minute;  
    inline constexpr auto h       = hour;  
    inline constexpr auto d       = day;  
  
} // namespace std::non_si::unit_symbols  
  
namespace std::si::unit_symbols {  
    using namespace non_si::unit_symbols;  
} // namespace std::si::unit_symbols
```

§ 6.2 Symbol Count

Category	Count
Base unit symbols (m, kg, s, A, K, mol, cd)	7
Named derived unit symbols	22
Prefixed base unit symbols (24 x 7)	168
Prefixed derived unit symbols (24 x 21)	504+
Non-SI unit symbols	14
Total	~715+

This is deliberately the largest component. For 21 of the 22 named derived SI units, all 24 SI prefixes are defined; `degree_Celsius` is excluded because prefixed Celsius temperatures have no physical meaning for an offset scale ($21 \times 24 = 504$). Units with non-ASCII symbols — Ω for ohm and μ for the micro prefix — additionally provide Unicode aliases, bringing the total above 504. The 14 non-SI unit symbols are comparatively modest.

The result is approximately **715+ named constants** in total.

This scale is an inherent property of SI: the standard explicitly sanctions every combination of its 24 prefixes with every base and derived unit. These names must exist for code such as `5 * MHz` or `300 * pm` to compile without resorting to verbose qualified expres-

sions like `5 * si::mega<si::hertz>`. LEWG should be aware of this magnitude when considering the unit symbols chapter.

§ 6.3 Why This Is a Separate Chapter

Unit symbols are a large, self-contained convenience layer. With ~715 short names they introduce distinct concerns — namespace pollution, compilation cost, and naming conflicts (e.g., `F`, `T`, `s`) — that merit independent consideration. LEWG may wish to accept the core unit definitions while deferring or declining this chapter.

Short, unqualified symbols can conflict with existing names in user or library code. For example, `F` clashes with a commonly used macro in some code bases, `T` can shadow template parameters, and `s` can conflict with string literals in some contexts. Making this feature opt-in (via a dedicated header and an explicit `using namespace std::si::unit_symbols;`) respects the longstanding C++ principle of not polluting namespaces by default.

The intended usage pattern is to include `<si_unit_symbols>` and apply the `using`-directive in `.cpp` implementation files. Header files that expose unit-aware interfaces should use the long-form names (`si::kilo<si::metre>`, `si::hertz`, etc.), as `using`-directives in headers are generally unwelcome. Projects that use only a handful of symbols may also be better served by defining them locally, e.g., `inline constexpr auto km = si::kilo<si::metre>;`.

Straw poll: *We want `<si_unit_symbols>` — the ~715+ short-form unit symbol aliases — to be included as part of the quantities and units library.*

§ 7 Optional: `std::chrono` Interoperability (`<si_chrono>`)

`<si_chrono>` provides bidirectional interoperability between SI time quantities and the `std::chrono` duration and time point types. This feature is only available in hosted environments (it requires `<chrono>`).

§ 7.1 Synopsis

```
namespace std {  
  
    // Customization point specialization enabling implicit construction of a  
    // quantity<si::second> (and prefixed variants) from a  
    // std::chrono::duration.  
    template<typename Rep, typename Period>  
    struct quantity_like_traits<chrono::duration<Rep, Period>> {  
        static constexpr auto reference = /* SI time unit derived from Period  
                                           */;  
        static constexpr bool explicit_import = false;  
        static constexpr bool explicit_export = false;  
    };  
}
```

```

using rep = Rep;
using T = chrono::duration<Rep, Period>;
[[nodiscard]] static constexpr rep to_numerical_value(const T& q) noexcept;
[[nodiscard]] static constexpr T from_numerical_value(const rep& v)
    noexcept;
};

// Absolute point origin associated with a clock's epoch
template<typename C>
struct chrono_point_origin_ : absolute_point_origin<isq::time> {
    using clock = C;
};

template<typename C>
inline constexpr chrono_point_origin_<C> chrono_point_origin;

// Customization point specialization enabling conversion between
// std::chrono::time_point and quantity_point with a clock-based origin.
template<typename C, typename Rep, typename Period>
struct quantity_point_like_traits<
    chrono::time_point<C, chrono::duration<Rep, Period>>> {
    static constexpr auto reference = /* SI time unit derived from Period */;
    static constexpr auto point_origin = chrono_point_origin<C>;
    static constexpr bool explicit_import = false;
    static constexpr bool explicit_export = false;
    using rep = Rep;
    using T = chrono::time_point<C, chrono::duration<Rep, Period>>;
    [[nodiscard]] static constexpr rep to_numerical_value(const T& tp) noexcept;
    [[nodiscard]] static constexpr T from_numerical_value(const rep& v)
        noexcept;
};

// Explicit export helpers
template<quantity_of<isq::duration> Q>
[[nodiscard]] constexpr auto to_chrono_duration(const Q& q);

template<quantity_point_of<isq::time> QP>
    requires is_specialization_of<decltype(QP::absolute_point_origin),
        chrono_point_origin_>
[[nodiscard]] constexpr auto to_chrono_time_point(const QP& qp);

} // namespace std

```

The period-to-unit mapping covers all standard `std::chrono` periods:

chrono instantiation	Mapped SI unit
<code>chrono::nanoseconds</code>	<code>nano<second></code>
<code>chrono::microseconds</code>	<code>micro<second></code>
<code>chrono::milliseconds</code>	<code>milli<second></code>
<code>chrono::seconds</code>	<code>second</code>
<code>chrono::minutes</code>	<code>minute</code>

chrono instantiation	Mapped SI unit
chrono::hours	hour
chrono::days	day
chrono::weeks	7 * day
chrono::duration<R, ratio<N,D>>	ratio<N,D> * second

The `quantity_like_traits` specialization with `explicit_export = false` allows implicit construction of a `chrono::duration` from a quantity, but only when the target type is already known at the call site:

```
chrono::seconds s = 5 * si::second;    // OK – target type known, implicit
                                     export
```

When the target chrono type must be *deduced* from the quantity’s unit (for example in a generic context, or when the unit is not one of the named chrono typedefs), the implicit path is unavailable because `std::ratio` cannot be computed without inspecting the unit’s canonical magnitude. The helper functions cover this case:

```
auto d = to_chrono_duration(750 * milli<second>);
// d is chrono::duration<int, ratio<1,1000>> – type deduced from the unit
//      magnitude

auto tp = to_chrono_time_point(qty_point);
// tp is chrono::time_point<Clock, ...> – clock and period both deduced
```

`to_chrono_time_point` additionally requires that the `quantity_point`’s absolute origin is a `chrono_point_origin_`, enforcing at compile time that the point is anchored to a clock epoch and not to an arbitrary physical reference.

Category	Count
Customization point specializations	2
Struct templates (<code>chrono_point_origin_</code>)	1
Variable template (<code>chrono_point_origin</code>)	1
Function templates (<code>to_chrono_*</code>)	2
Total	6

This introduces **2 struct templates**, **1 variable template**, **2 function templates**, and **2 specializations** of library customization points.

§ 7.2 Why This Is a Separate Chapter

`std::chrono` interoperability is a distinct integration concern — it bridges two independent library features and depends on `<chrono>`, a hosted-only facility. Separating it into its own chapter lets LEWG vote on this integration independently and allows the mandatory core to remain usable in freestanding environments.

Straw poll: We want the `std::chrono` interoperability type traits (`quantity_like_traits` , `quantity_point_like_traits` , `chrono_point_origin_`) to be included as part of the quantities and units library.

Straw poll: We want the explicit `to_chrono_duration` and `to_chrono_time_point` conversion helpers to be included as part of the quantities and units library.

§ 8 Optional: Mathematical Functions for SI Angles (`<si_math>`)

`<si_math>` provides overloads of the standard trigonometric functions in the `std::si` namespace. These overloads accept and return properly-typed SI angular quantities rather than plain numerical values. This feature is only available in hosted environments (it requires `<cmath>`).

§ 8.1 Motivation

The standard `<cmath>` functions `sin` , `cos` , `tan` , etc. operate on raw `double` values representing radians. When using a quantities library, it is desirable to have overloads that:

- Accept `quantity<isq::angular_measure, R>` for any angular unit `R` , performing automatic conversion to radians before calling the underlying `std::sin` .
- Return a dimensionless quantity (for the forward functions) or a quantity `<isq::angular_measure, radian>` (for the inverse functions).

This ensures that, for example, `si::sin(30 * deg)` yields the same result as `std::sin( $\pi/6$ )` , without requiring the user to manually convert degrees to radians.

§ 8.2 Synopsis

```
namespace std::si {

template<reference_of<isq::angular_measure> auto R, typename Rep>
    requires requires(Rep v) { sin(v); } || requires(Rep v) { std::sin(v); }
[[nodiscard]] quantity_of<dimensionless> auto sin(const quantity<R, Rep>&
    q) noexcept;

template<reference_of<isq::angular_measure> auto R, typename Rep>
    requires requires(Rep v) { cos(v); } || requires(Rep v) { std::cos(v); }
[[nodiscard]] quantity_of<dimensionless> auto cos(const quantity<R, Rep>&
    q) noexcept;

template<reference_of<isq::angular_measure> auto R, typename Rep>
    requires requires(Rep v) { tan(v); } || requires(Rep v) { std::tan(v); }
[[nodiscard]] quantity_of<dimensionless> auto tan(const quantity<R, Rep>&
    q) noexcept;

template<reference_of<dimensionless> auto R, typename Rep>
```

```

    requires requires(Rep v) { asin(v); } || requires(Rep v) { std::asin(v);
    }
[[nodiscard]] quantity_of<isq::angular_measure> auto asin(const quanti-
    ty<R, Rep>& q) noexcept;

template<reference_of<dimensionless> auto R, typename Rep>
    requires requires(Rep v) { acos(v); } || requires(Rep v) { std::acos(v);
    }
[[nodiscard]] quantity_of<isq::angular_measure> auto acos(const quanti-
    ty<R, Rep>& q) noexcept;

template<reference_of<dimensionless> auto R, typename Rep>
    requires requires(Rep v) { atan(v); } || requires(Rep v) { std::atan(v);
    }
[[nodiscard]] quantity_of<isq::angular_measure> auto atan(const quanti-
    ty<R, Rep>& q) noexcept;

template<auto R1, typename Rep1, auto R2, typename Rep2>
    requires get_common_reference(R1, R2) &&
        (requires(Rep1 v1, Rep2 v2) { atan2(v1, v2); } || requires(Rep1
        v1, Rep2 v2) { std::atan2(v1, v2); })
[[nodiscard]] quantity_of<isq::angular_measure> auto atan2(const quanti-
    ty<R1, Rep1>& y, const quantity<R2, Rep2>& x) noexcept;

} // namespace std::si

```

Category	Count
Forward trig (sin, cos, tan)	3
Inverse trig (asin, acos, atan)	3
Two-argument (atan2)	1
Total	7

This introduces **7 function templates** in `std::si`.

§ 8.3 Why This Is a Separate Chapter

Trigonometric overloads for SI angles are a self-contained addition. They depend on `<cmath>` (a hosted-only facility) and concern a specific quantity kind (`angular_measure`); other math overloads such as `sqrt` , `pow` , and `abs` belong to the general quantities framework rather than SI. Separating them lets LEWG vote on this feature independently.

Straw poll: *We want `<si_math>` — trigonometric function overloads for SI angular quantities — to be included as part of the quantities and units library.*

§ 9 Optional: Prefix Utilities (`<si_prefix_utils>`)

`<si_prefix_utils>` provides a utility for automatically selecting the most appropriate SI prefix for a quantity. This feature is only available in hosted environments (it requires `<cmath>` for `log10` , `floor` , and `abs`).

§ 9.1 Motivation

A common need is to express a quantity using the prefix that keeps the numerical value in a human-friendly range — whether for display, logging, or passing to another function. For example, normalizing 0.003 m to 3 mm, or 1 500 000 Hz to 1.5 MHz. The `invoke_with_prefixed` facility automates prefix selection and delegates the actual use of the rescaled quantity to a user-supplied callable, making it useful wherever a “best-prefix” quantity is needed, not only for formatting output.

§ 9.2 Synopsis

```
namespace std::si {

enum class prefix_range : uint8_t {
    engineering, // selects only powers of 1000 (milli, kilo, mega, ...)
                  // resulting in a value in the range [1.0, 1000)
    full         // selects all 24 prefixes (including deca, hecto, deci,
                  // value in [1.0, 10) within the milli-kilo range, [1.0,
                  // 1000) elsewhere
};

/**
 * Calls func with q rescaled to the SI prefix that gives the numerical
 * value
 * at least min_integral_digits digits in its integral part.
 */
template<Quantity Q, invocable<Q> Func, PrefixableUnit U, auto Character
        = quantity_character::real_scalar>
    requires RepresentationOf<typename Q::rep, Character> && treat_as_floating_point<typename Q::rep> &&
        requires(typename Q::rep v) {
        requires requires { abs(v); } || requires { std::abs(v);
    };
        requires requires { log10(v); } || requires { std::log10(v);
    };
        requires requires { floor(v); } || requires { std::floor(v);
    };
    }
constexpr decltype(auto) invoke_with_prefixed(Func func, Q q, U u,
                                              prefix_range range =
                                              prefix_range::engineering,
                                              int min_integral_digits =
                                              1);

} // namespace std::si
```

Usage example:

```
using namespace si::unit_symbols;
invoke_with_prefixed([](auto q){ std::print("{} ", q); }, 0.003 * m,
    metre);
// prints "3 mm"
```

```
invoke_with_prefixed([](auto q){ std::print("{} ", q); }, 1.5e6 * Hz,
    hertz);
// prints "1.5 MHz"

invoke_with_prefixed([](auto q){ std::print("{} ", q); }, 42 * km, metre,
    prefix_range::engineering, 2);
// prints "42 km" (at least 2 integral digits requested)
```

Category	Count
Scoped enumeration (prefix_range)	1
Function template (invoke_with_prefixed)	1
Total	2

This introduces **1 scoped enumeration** and **1 function template** in `std::si`.

§ 9.3 Why This Is a Separate Chapter

Auto-prefix selection is a convenience utility — distinct in kind from SI definitions — for obtaining a quantity rescaled to the most human-friendly prefix. It depends on `<cmath>` and floating-point arithmetic. Separating it into its own chapter lets LEWG vote on this feature independently of the quantity and unit definitions.

Straw poll: *We want `<si_prefix_utils>` — the `invoke_with_prefixed` auto-prefix selection utility — to be included as part of the quantities and units library.*

§ 10 Summary

§ 10.1 Definition Counts by Component

Component	Header	Type	Definitions
ISQ base dimensions	<code><isq_si_quantities></code>	mandatory	7
ISQ base quantities (incl. time alias)	<code><isq_si_quantities></code>	mandatory	8
ISQ derived quantities for SI (incl. aliases)	<code><isq_si_quantities></code>	mandatory	26
ISQ subtotal	<code><isq_si_quantities></code>		41
SI prefixes (class + variable templates)	<code><si_core></code>	mandatory	48
SI base units	<code><si_core></code>	mandatory	8
SI named derived units and origins	<code><si_core></code>	mandatory	25
SI core subtotal	<code><si_core></code>		81
ISQ + SI core total			122

Component	Header	Type	Definitions
Non-SI units accepted for use with SI	<code><si_accepted_units></code>	optional	13
Customization point specializations	<code><si_accepted_units></code>	optional	3
SI-defining constants (si2019 + derived)	<code><si_constants></code>	optional	10
Unit symbols (SI + non-SI)	<code><si_unit_symbols></code>	optional	~715+
<code>std::chrono</code> interoperability	<code><si_chrono></code>	hosted only	~6
Trigonometric math functions	<code><si_math></code>	hosted only	7
Prefix auto-selection utility	<code><si_prefix_utils></code>	hosted only	2
Optional subtotal			~756+
Grand total			~878+

§ 11 Consolidated Straw Polls

Poll 1 (`<isq_si_quantities>`): *We want `<isq_si_quantities>` — the minimal ISQ subset (41 definitions) required by `<si_core>` — to be standardized as part of the quantities and units library.*

Poll 2 (`<si_core>`): *We want `<si_core>` — 81 SI definitions including 24 prefixes, 8 base units, and 25 derived units and origins — to be standardized as part of the quantities and units library.*

Poll 3 (`<si_accepted_units>`): *We want `<si_accepted_units>` — non-SI units accepted for use with SI — to be included as part of the quantities and units library.*

Poll 4a (`<si_constants>` , defining): *We want the seven SI-defining constants (`si2019` namespace) to be included as part of the quantities and units library.*

Poll 4b (`<si_constants>` , derived): *We want a selected set of derived physical constants to be included as part of the quantities and units library.*

Poll 5 (`<si_unit_symbols>`): *We want `<si_unit_symbols>` — the ~720+ short-form unit symbol aliases — to be included as part of the quantities and units library.*

Poll 6a (`<si_chrono>` , traits): *We want the `std::chrono` interoperability type traits (`quantity_like_traits` , `quantity_point_like_traits` , `chrono_point_origin_`) to be included as part of the quantities and units library.*

Poll 6b (`<si_chrono>` , helpers): *We want the explicit `to_chrono_duration` and `to_chrono_time_point` conversion helpers to be included as part of the quantities and units library.*

Poll 7 (`<si_math>`): *We want `<si_math>` — trigonometric function overloads for SI angular quantities — to be included as part of the quantities and units library.*

Poll 8 (`<si_prefix_utils>`): *We want `<si_prefix_utils>` — the `invoke_with_pre-fixed` auto-prefix selection utility — to be included as part of the quantities and units library.*

§ 12 Acknowledgements

Special thanks and recognition goes to [The C++ Alliance](#) for supporting Mateusz's membership in the ISO C++ Committee and the production of this proposal.

§ 13 References

[P3045R8] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Chip Hogg, Nicolas Holthaus, Roth Michaels, and Vincent Reverdy. Quantities and units library.

<https://wg21.link/p3045r8>

[P4185R0] Mateusz Pusz. Completing the Mathematical Model for C++ Quantities and Units.

<https://wg21.link/p4185r0>

[SI] SI Brochure: The International System of Units (SI).

<https://www.bipm.org/en/publications/si-brochure>